Lezione 5 – Concetti OO, classi in Java

I concetti della programmazione a oggetti e le classi in Java

Mattia Fonisto

Laboratorio di Programmazione – Ingegneria Informatica, II anno
Università degli Studi di Napoli Federico II – Sede di San Giovanni a Teduccio

13 ottobre 2026

Agenda della lezione (2h)

1. **Ripasso e Homework 4** (*15 min*)
   - uno di voi alla lavagna con `Anagrammi`, `Pagella` o `Cifrario`: lo commentiamo insieme
2. **Programmazione a oggetti: i concetti** (*15 min*)
   - tipo di dato astratto, classi e oggetti, relazioni tra classi, i tre principi
3. **Classi in Java** (*30 min*)
   - dichiarare e usare una classe: variabili di istanza, metodi, `public` e `private`
   - ciclo di vita di un oggetto, costruttori, metodi `get` e `set`, predicati, garbage collector
4. **Esercitazione** – tutti insieme (*50 min*)
   - *Commentiamo insieme*: `Studente`, una classe completa e il suo programma di prova
   - *Insieme, passo passo*: `ContoBancario` – dal programma di prova alla classe
5. **Wrap-up e Homework 5** (*10 min*)

Programmazione a oggetti: il tipo di dato astratto

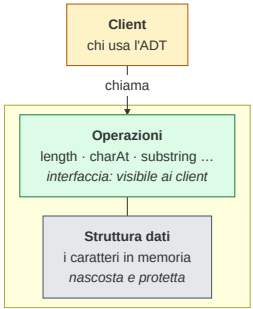
Tipo di dato astratto (ADT, *Abstract Data Type*)

Una struttura dati *incapsulata*, cioè racchiusa, dalle operazioni consentite su di essa: ai dati si accede, in lettura e in scrittura, soltanto attraverso quelle operazioni.

Ne usate uno dalla lezione 2: `String`. Conoscete le sue operazioni (`length`, `charAt`...), non il modo in cui tiene in memoria i caratteri. Un ADT è il modulo che realizza un'*astrazione sui dati* (lezione 4).

- l'**interfaccia** è l'elenco delle operazioni: la sola parte che i client usano
- il **corpo**, o *realizzazione*, è la struttura dati con il codice delle operazioni: lo scrive chi sviluppa l'ADT

Che cosa si guadagna: nessuna operazione scorretta può alterare la struttura dati, e la realizzazione si può modificare senza toccare i client.



Come chi guida un'automobile: usa volante, pedali e cambio, non mette le mani nel motore.

Java ha anche una parola chiave `abstract`: un costrutto specifico (lezione 7).

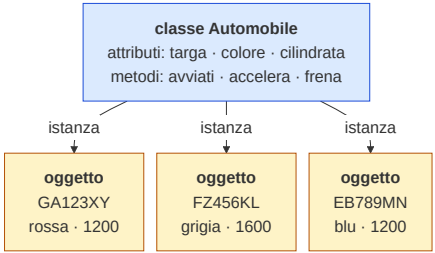
Programmazione a oggetti: classi e oggetti

Classe

Descrive un gruppo di oggetti con le stesse proprietà, gli *attributi*, e lo stesso comportamento, i *metodi*. È lo schema da cui gli oggetti vengono creati.

Oggetto

Un'*istanza*, cioè un esemplare, di una classe. I valori dei suoi attributi sono il suo *stato*.



`String` è una classe; `"ciao"` e `"Java"` sono due suoi oggetti: stessi metodi, caratteri diversi.

- due oggetti della stessa classe si distinguono soltanto per lo stato: il comportamento è identico
- quali attributi servono dipende da chi usa la classe: sceglierli è un'astrazione (lezione 4)

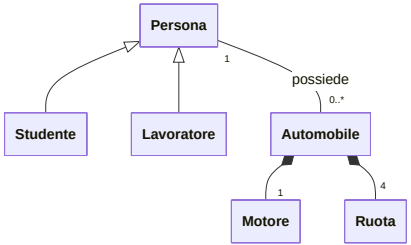
Programmare a oggetti: si individuano le classi di oggetti del problema, si realizza ciascuna come un ADT (gli attributi sono la struttura dati, i metodi le operazioni) e si costruisce il programma assemblando oggetti che collaborano, chiamando l'uno i metodi dell'altro.

Programmazione a oggetti: relazioni tra classi

Le classi di un programma non sono isolate: fra i loro oggetti esistono tre tipi di relazione.

- **Composizione**: un oggetto complesso è composto da oggetti più semplici, i suoi *componenti*. Ogni automobile è composta da un motore e da quattro ruote.
- **Generalizzazione-specializzazione**: una classe è un caso particolare di un'altra. Uno studente e un lavoratore *sono* persone: hanno in comune il comportamento di ogni persona, e ciascuno vi aggiunge il proprio.
- **Associazione**: gli oggetti di due classi sono collegati, senza che gli uni siano parte degli altri. Una persona possiede una o più automobili; un libro ha più autori, e un autore ha scritto più libri.

Nella lezione 4 il lettore musicale aveva già due di queste relazioni: la playlist *è composta da* brani (composizione) e *usa* il riproduttore audio (associazione).



Il disegno è un diagramma **UML** (*Unified Modeling Language*), il linguaggio grafico più usato per descrivere classi e relazioni. Il triangolo indica la generalizzazione, e punta alla classe più generale; il rombo pieno indica la composizione, e sta dalla parte del composto; la linea semplice indica l'associazione. I numeri dicono quanti oggetti partecipano.

Programmazione a oggetti: i tre principi

Incapsulamento

Il meccanismo che consente l'accesso a una struttura dati soltanto attraverso le sue operazioni. È ciò che fa di una classe un ADT. *oggi*

Ereditarietà

Il meccanismo con cui una classe acquisisce le proprietà di un'altra. Realizza la generalizzazione-specializzazione: `Studente` eredita da `Persona`. *lezione 7*

Polimorfismo

Un'interfaccia, più realizzazioni: si chiama un metodo senza sapere, fino all'esecuzione, su quale tipo di oggetto agirà. Con `f.area()` un cerchio calcola πr^2, un rettangolo base per altezza: decide l'oggetto a cui `f` si riferisce. *lezione 7*

Tutti i linguaggi a oggetti offrono questi tre meccanismi.

Che cosa si guadagna, con le parole della lezione 4:

- **modularità**: le classi sono i moduli
- **coesione**: una classe, una sola entità
- **basso accoppiamento**: i metodi lavorano sui dati del proprio oggetto
- **information hiding**: dati e algoritmi restano nascosti nell'oggetto
- **riuso**: con l'ereditarietà
- **estensibilità**: con il polimorfismo

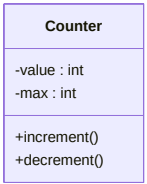
Dichiarare una classe in Java – `Counter.java` (versione 1)

```java
public class Counter {

    private int value;    // valore corrente
    private int max;      // valore massimo

    public void increment() {
        value++;
    }

    public void decrement() {
        value--;
    }
}
```



Il costrutto `class` definisce un nuovo tipo di dato: la struttura dati e le operazioni consentite su di essa.

- **variabili di istanza**: sono gli attributi. Ogni oggetto ha le proprie, e i loro valori sono il suo stato
- **metodi**: sono *senza* `static`, e lavorano sulle variabili di istanza dell'oggetto su cui vengono chiamati
- `private`: le variabili di istanza si usano solo dentro la classe. È l'incapsulamento. Nella lezione 4 `private` nascondeva un metodo; qui nasconde i dati
- `public`: l'interfaccia, cioè le operazioni a disposizione dei client
- una classe `public` per file, con lo stesso nome: `Counter.java`

In UML una classe è un rettangolo in tre sezioni: nome, variabili di istanza, metodi. Il segno – indica `private`, il segno + indica `public`.

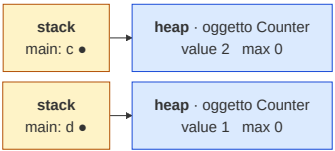
Usare una classe – ProvaCounter.java (versione 1)

Nel main di ProvaCounter:

```
Counter c = new Counter();
Counter d = new Counter();

c.increment();
c.increment();
c.increment();
c.decrement();
d.increment();

// ERRORE: value e' private
// c.value = 100;
// System.out.println(d.value);
```

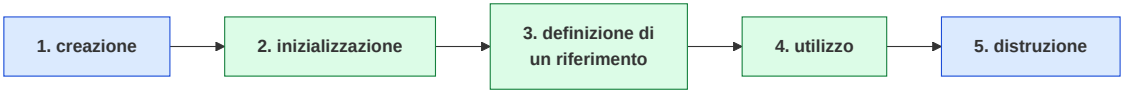


- `new Counter()` crea un oggetto nello heap e ne restituisce il riferimento, come `new int[3]` e `new Scanner(System.in)`
- ogni oggetto ha le **sue** variabili di istanza: `c` e `d` contano separatamente. Nascono con il valore di default (`0`, `0.0`, `false`, `null`), come gli elementi di un array; le variabili locali, invece, vanno inizializzate
- un metodo non `static` si chiama **su un oggetto**: `c.increment()`. Finora scrivevamo `Voti.isValido(31)`, sul nome della classe, perché un metodo `static` non ha un oggetto su cui lavorare
- `Counter` non ha un `main`: da sola non è un programma. `ProvaCounter`, nella stessa cartella, è un suo client

Così com'è, però, `Counter` è inutile: non possiamo scegliere `max`, né sapere quanto vale `value`.

Ciclo di vita di un oggetto

Una classe descrive soltanto la struttura e le operazioni comuni a tutti i suoi oggetti. Un oggetto esiste dopo un'operazione di *istanziazione*, che lo crea e lo alloca in memoria; da quel momento attraversa cinque fasi.



Fase	**In** `Counter c = new Counter(4); c.increment();`	**Chi se ne occupa**
1. creazione	`new` riserva nello heap lo spazio per le variabili di istanza	la JVM
2. inizializzazione	`Counter(4)` chiama un **costruttore** della classe	chi scrive la classe
3. riferimento	`Counter c =` conserva il riferimento restituito da `new`	il client
4. utilizzo	`c.increment()`: i metodi chiamati attraverso il riferimento	il client
5. distruzione	quando nessun riferimento porta più all'oggetto	la JVM

I costruttori – `Counter.java` (versione 2)

Aggiunti alla versione 1:

```
// costruttore senza argomenti
public Counter() {
    max = 10;
    value = 0;
}

// costruttore con argomenti
public Counter(int m) {
    max = m;
    value = 0;
}

// costruttore di copia
public Counter(Counter c) {
    max = c.max;
    value = c.value;
}
```

Nel costruttore di copia si legge `c.max`, benché `max` sia `private`. È lecito: una variabile `private` è **visibile in tutta la classe** che la dichiara. Il costruttore fa parte di `Counter` e `c` è un `Counter`: vede le variabili di `c` come vede le proprie.

Un **costruttore** è un metodo speciale che inizializza l'oggetto appena creato. Lo chiama `new`.

- ha lo **stesso nome della classe** e **nessun tipo restituito**, nemmeno `void`
- una classe può averne più di uno, distinti per *overloading*, cioè per numero e tipo dei parametri (lezione 3): **senza argomenti**, **con argomenti**, **di copia**. Quest'ultimo ha per parametro un oggetto della stessa classe
- se una classe non ha costruttori, il compilatore ne fornisce uno **di default**, senza argomenti, che lascia nelle variabili i valori di default: è quello usato dalla versione 1
- **attenzione**: basta definire un costruttore, e quello di default non viene più fornito. Senza il primo dei tre, `new Counter()` non compilerebbe

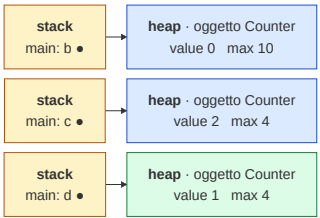
Tre modi di creare un contatore – `ProvaCounter.java` (versione 2)

Nel `main` di `ProvaCounter`:

```
Counter b = new Counter();     // max 10
Counter c = new Counter(4);    // max 4
c.increment();

Counter d = new Counter(c);    // copia di c
c.increment();                 // d non cambia

// ERRORE: max e' private, non e' visibile
// fuori dalla classe Counter
// System.out.println(c.max);
```



- `new` sceglie il costruttore in base agli argomenti, come in ogni chiamata a metodi in overloading
- il **costruttore di copia** riceve un riferimento a un oggetto già esistente e ne copia lo stato nel nuovo oggetto, una variabile alla volta. Da quel momento i due oggetti sono indipendenti: il secondo `c.increment()` non tocca `d`
- la visibilità `private` è **per classe**, non per oggetto. Dentro `Counter`, `max` è visibile per ogni oggetto `Counter`: quello che si sta costruendo e `c`. Alle altre classi `max` non è visibile: nel `main` di `ProvaCounter`, `c.max` non compila

Ora scegliamo lo stato iniziale; non sappiamo ancora leggere `value`.

I metodi get e set – Counter.java (versione 3)

Aggiunti alla versione 2:

```
    // metodi get: leggono lo stato
    public int getValue() {
        return value;
    }

    public int getMax() {
        return max;
    }

    // metodo set: modifica lo stato
    public void setMax(int m) {
        max = m;
    }
```

- un metodo **get** restituisce il valore di una variabile di istanza; un metodo **set** lo sostituisce con quello ricevuto
- è la classe a decidere che cosa si può leggere e che cosa si può modificare: `value` ha solo il get, e cambia soltanto con `increment` e `decrement`

Nel `main` di `ProvaCounter`:

```
Counter b = new Counter();
Counter c = new Counter(4);
c.increment();
Counter d = new Counter(c);
c.increment();

System.out.println(b.getMax());      // 10
System.out.println(c.getValue());    // 2
System.out.println(d.getValue());    // 1

d.setMax(20);
System.out.println(d.getMax());      // 20
System.out.println(c.getMax());      // 4

// c.value = 3;      // ERRORE: private
// c.setValue(3);    // ERRORE: non esiste
```

I nomi: `get` o `set`, seguito dal nome della variabile con l'iniziale maiuscola.

Predicati e precondizioni – Counter.java (versione 4)

Aggiunti alla versione 3:

```
// predicati: true se l'operazione
// porterebbe value fuori da 0..max
public boolean ovf() {
    return (value + 1) > max;
}

public boolean unf() {
    return (value - 1) < 0;
}
```

- un **predicato** è un metodo che restituisce un `boolean` sullo stato dell'oggetto: si usa nelle condizioni di `if` e `while`
- `ovf` (*overflow*) e `unf` (*underflow*) verificano le **precondizioni** di `increment` e `decrement`: il valore non deve superare `max` né scendere sotto zero
- qui il controllo spetta al **client**: se dimentica l'`if`, `increment` porta `value` oltre `max`. Nell'esercitazione lo sposteremo dentro i metodi

Nel `main` di `ProvaCounter`:

```
Counter c = new Counter(2);
while (!c.ovf()) {
    c.increment();
}
System.out.println(c.getValue());   // 2

if (!c.ovf()) {
    c.increment();
} else {
    System.out.println("c al massimo");
}

c.decrement();
c.decrement();
if (!c.unf()) {
    c.decrement();
} else {
    System.out.println("c a zero");
}
System.out.println(c.getValue());   // 0
```

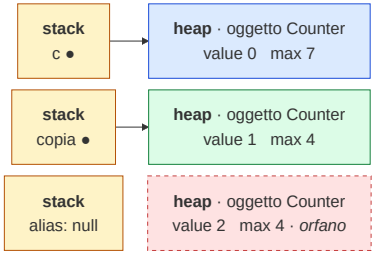
Copia, alias e distruzione degli oggetti – `Riferimenti.java`

```
Counter c = new Counter(4);
c.increment();

Counter copia = new Counter(c);  // altro oggetto
Counter alias = c;               // stesso oggetto
c.increment();
System.out.println(c.getValue());       // 2
System.out.println(copia.getValue());   // 1
System.out.println(alias.getValue());   // 2

c = new Counter(7);    // c cambia oggetto
alias = null;          // il primo e' orfano
System.out.println(c.getValue());       // 0
```

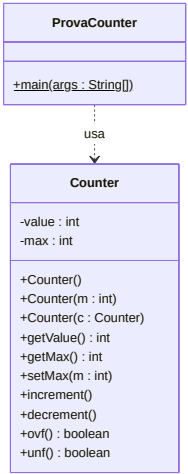
- `alias = c` copia il **riferimento**, non l'oggetto: vale per ogni oggetto, come per gli array della lezione 3. Per avere un secondo oggetto con lo stesso stato serve il costruttore di copia
- un oggetto vive finché almeno un riferimento porta a lui



La memoria dopo l'ultima riga.

Un oggetto *orfano*, cioè senza più riferimenti, non è raggiungibile da nessuna istruzione. Lo dealloca automaticamente il **garbage collector** della JVM: in Java la distruzione degli oggetti non è responsabilità del programmatore.

La classe come modulo: client e interfaccia



Una classe è un modulo, e ha due ruoli possibili: **client**, quando usa un'altra classe, e classe **usata** da un client. `ProvaCounter` è client di `Counter`.

I due ruoli non si escludono: la stessa classe può essere client di una classe ed essere usata da un'altra. Nella lezione 4 `Statistiche` era client di `Arrays` e, allo stesso tempo, era usata da `Frequenze`.

Per usare una classe, un client dichiara un riferimento (`Counter c`), crea un'istanza con `new` e chiama i metodi `public`. Se le due classi stanno nella stessa cartella non serve alcun `import`.

Lo stato dell'oggetto è incapsulato: cambia soltanto per effetto dei metodi chiamati sull'oggetto.

L'interfaccia pubblica di una classe deve essere:

- **stabile**: cambiarla significa riscrivere i client
- **minimale**: espone soltanto ciò che serve ai client
- **completa**: offre tutte le operazioni che servono ai client

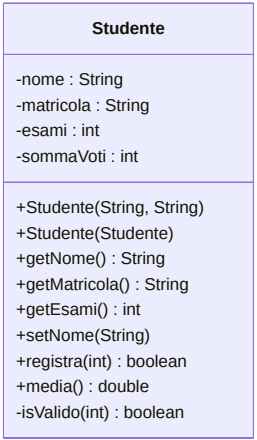
Commentiamo insieme (20 min) – il problema

Il problema. La segreteria deve tenere traccia della carriera di ogni studente: il nome, la matricola, quanti esami ha superato e con quale media.

Le regole

- uno studente nasce con un nome e una matricola, e senza esami
- la matricola non cambia mai; il nome si può correggere
- un esame si registra con il suo voto, da 18 a 30; ogni altro voto viene rifiutato
- il numero degli esami e la media cambiano soltanto registrando un esame
- di uno studente si può creare una copia, indipendente dall'originale

La soluzione è già scritta: la classe `Studente`, che fa rispettare le regole, e il programma `ProvaStudente`, che la usa. Le leggiamo insieme, riga per riga, distinguendo interfaccia e corpo.



Commentiamo insieme – Studente.java (1/2)

```java
/** Uno studente e i suoi esami. */
public class Studente {

    private String nome;
    private String matricola;
    private int esami;        // quanti esami
    private int sommaVoti;    // somma dei voti

    /** Un nuovo studente, senza esami. */
    public Studente(String n, String m) {
        nome = n;
        matricola = m;
        esami = 0;
        sommaVoti = 0;
    }

    /** Costruttore di copia. */
    public Studente(Studente s) {
        nome = s.nome;
        matricola = s.matricola;
        esami = s.esami;
        sommaVoti = s.sommaVoti;
    }
```

```java
    public String getNome() {
        return nome;
    }

    public String getMatricola() {
        return matricola;
    }

    public int getEsami() {
        return esami;
    }

    public void setNome(String n) {
        nome = n;
    }
```

- nessun costruttore senza argomenti: uno studente senza nome e senza matricola non ha senso, e `new Studente()` non compila
- tre get, un solo set: la matricola non cambia mai, e gli esami non si scrivono "a mano"

Commentiamo insieme – `Studente.java` (2/2)

```
    /** false se il voto non e' valido. */
    public boolean registra(int voto) {
        if (!isValido(voto)) {
            return false;
        }
        esami++;
        sommaVoti += voto;
        return true;
    }

    /** La media; 0 se non ci sono esami. */
    public double media() {
        if (esami == 0) {
            return 0;
        }
        return (double) sommaVoti / esami;
    }

    // corpo: visibile solo dentro Studente
    private boolean isValido(int voto) {
        return voto >= 18 && voto <= 30;
    }
}
```

Da notare, leggendo:

- `registra` è l'unico modo di cambiare `esami` e `sommaVoti`: le due variabili restano coerenti fra loro, e un voto non valido non entra mai nello stato
- `registra` restituisce un `boolean`: il client sa se l'operazione è riuscita. In `Counter` il controllo era del client (`ovf`); qui è **dentro il metodo**
- `media` non è una variabile di istanza: si calcola dallo stato quando serve, con il cast della lezione 4
- `isValido` è `private`: appartiene al corpo. `registra` lo chiama con il solo nome, sullo stesso oggetto
- nel costruttore di copia `nome = s.nome` copia un riferimento: i due oggetti condividono la stessa `String`, che però è immutabile

Commentiamo insieme – ProvaStudente.java

```java
/** LEGGE lo studente ricevuto e lo stampa. */
public static void stampa(Studente s) {
    System.out.println(s.getNome()
            + ": esami " + s.getEsami()
            + ", media " + s.media());
}

public static void main(String[] args) {
    Studente a = new Studente("Giulia", "N46001234");
    a.registra(28);
    a.registra(24);
    System.out.println(a.registra(31));
    stampa(a);

    Studente b = a;
    Studente c = new Studente(a);
    b.registra(29);
    c.registra(26);
    c.setNome("Giulia Russo");
    stampa(a);
    stampa(b);
    stampa(c);
}
```

```
false
Giulia: esami 2, media 26.0
Giulia: esami 3, media 27.0
Giulia: esami 3, media 27.0
Giulia Russo: esami 3, media 26.0
```

Rispondiamo insieme:

1. quanti oggetti Studente esistono alla fine del main? E quanti riferimenti?
2. registra(29) è chiamato su b: perché cambia anche la riga stampata per a?
3. stampa riceve una copia del riferimento: potrebbe modificare lo studente del main? Con quali istruzioni?
4. possiamo scrivere qui a.esami = 30? E a.isValido(25)?
5. che cosa restituisce media per uno studente appena creato? A che serve il suo if?

Insieme, passo passo (30 min) – il problema e `ProvaConto.java`

Il problema. Un conto bancario ha un intestatario, un saldo e il numero dei movimenti riusciti. Il saldo non deve mai diventare negativo. Il programma di prova è già scritto: manca la classe `ContoBancario`.

```java
/** LEGGE il conto ricevuto e lo stampa su una riga. */
public static void stampa(ContoBancario c) {
    System.out.println(c.getIntestatario()
            + ": saldo " + c.getSaldo()
            + ", movimenti " + c.getMovimenti());
}

public static void main(String[] args) {
    ContoBancario anna = new ContoBancario("Anna", 500);
    ContoBancario luca = new ContoBancario("Luca");

    anna.versa(250.50);
    System.out.println(anna.preleva(1000));
    System.out.println(anna.versa(-20));
    System.out.println(anna.trasferisci(luca, 300));
    System.out.println(luca.trasferisci(anna, 999));

    stampa(anna);
    stampa(luca);
}
```

Deve stampare:

```
false
false
true
false
Anna: saldo 450.5, movimenti 3
Luca: saldo 300.0, movimenti 1
```

Il client viene prima della classe: leggendolo ricaviamo l'**interfaccia** che `ContoBancario` deve offrire.

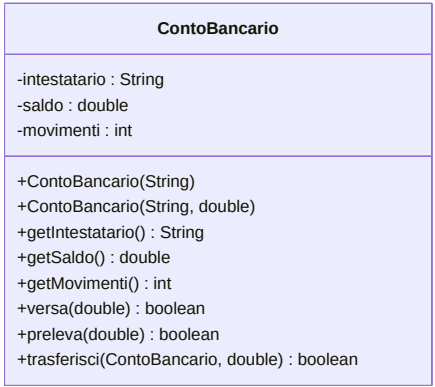
`ProvaConto` non contiene un solo `if`: che un'operazione sia lecita lo decide il conto, e lo comunica con il valore restituito.

Insieme, passo passo – dal programma di prova alla classe

Prima di scrivere codice, decidiamo insieme:

1. quali **variabili di istanza**? → `intestatario`, `saldo`, `movimenti`: tutte `private`
2. in quanti modi nasce un conto? → due **costruttori**: solo l'intestatario, oppure l'intestatario e un primo versamento
3. quali `get`, quali `set`? → tre `get`. Nessun `setSaldo`, nessun `setMovimenti`: il saldo cambia soltanto con `versa`, `preleva` e `trasferisci`
4. chi controlla gli importi? → i **metodi** stessi: restituiscono `false`, e non toccano lo stato, se l'importo non è positivo o supera il saldo
5. come si scrive `trasferisci`? → con `preleva` su questo conto e `versa` sull'altro: i controlli ci sono già

Da tenere a mente: nessuna sequenza di chiamate, per quanto sbagliata, deve poter portare il saldo sotto zero.



Una possibile soluzione (1/2) – ContoBancario.java

```java
/** Un conto corrente bancario. */
public class ContoBancario {

    private String intestatario;
    private double saldo;
    private int movimenti;   // operazioni riuscite

    /** Un conto vuoto. */
    public ContoBancario(String nome) {
        intestatario = nome;
        saldo = 0;
        movimenti = 0;
    }

    /** Un conto aperto con un primo versamento. */
    public ContoBancario(String nome, double primo) {
        intestatario = nome;
        saldo = 0;
        movimenti = 0;
        versa(primo);
    }

    public String getIntestatario() {
        return intestatario;
    }

    public double getSaldo() {
        return saldo;
    }

    public int getMovimenti() {
        return movimenti;
    }
```

- il secondo costruttore non scrive `saldo = primo`: chiama `versa`, che controlla l'importo e conta il movimento
- `saldo` ha un get e nessun set: il client lo legge quando vuole, ma non lo scrive
- i parametri hanno nomi diversi dalle variabili di istanza: con lo stesso nome, il parametro nasconderebbe la variabile (lezione 6)

Una possibile soluzione (2/2) – ContoBancario.java

```
    /** false se l'importo non e' positivo. */
    public boolean versa(double importo) {
        if (importo <= 0) {
            return false;
        }
        saldo += importo;
        movimenti++;
        return true;
    }
```

```
    /** false se l'importo non e' positivo
     *  o supera il saldo. */
    public boolean preleva(double importo) {
        if (importo <= 0 || importo > saldo) {
            return false;
        }
        saldo -= importo;
        movimenti++;
        return true;
    }
```

```
    /** false se il prelievo da questo conto fallisce. */
    public boolean trasferisci(ContoBancario altro, double importo) {
        if (!preleva(importo)) {
            return false;
        }
        altro.versa(importo);
        return true;
    }
}
```

`trasferisci` lavora su due oggetti: `preleva` agisce sul conto su cui il metodo è chiamato, `altro.versa` sul conto ricevuto. `altro` è una copia del riferimento: il versamento modifica l'oggetto del chiamante.

Ogni metodo controlla **prima** di modificare: se restituisce `false`, lo stato è rimasto com'era.

Riepilogo: cosa abbiamo trattato

Concetto (unità di riferimento)	**Dove compare oggi**
Tipo di dato astratto, interfaccia e realizzazione (U4)	`String`; ogni classe scritta oggi
Classi, oggetti, stato (U4)	`Counter` e i suoi oggetti `b`, `c`, `d`
Composizione, gen-spec, associazione; UML (U4)	i diagrammi di `Persona`, `Automobile`, `Counter`
Incapsulamento, ereditarietà, polimorfismo (U4)	l'incapsulamento: `private` in tutte le classi
Dichiarazione di classe, variabili di istanza, metodi (U5)	`Counter` versione 1
`new`, riferimenti, chiamata su un oggetto (U5)	`ProvaCounter`; `ProvaStudente`
Ciclo di vita; costruttori e loro overloading (U5)	`Counter` versione 2; `Studente`; `ContoBancario`
Costruttore di copia, alias (U5)	`Riferimenti`; `b` e `c` in `ProvaStudente`
Metodi `get` e `set` (U5)	`Counter` versione 3; `setNome`; nessun `setSaldo`
Predicati e precondizioni (U5)	`ovf`, `unf`; `registra`, `versa`, `preleva`
Distruzione degli oggetti, garbage collector (U5)	`Riferimenti`
Client e classe usata; interfaccia pubblica (U5)	`ProvaConto`, scritto prima di `ContoBancario`

Prossima lezione (L6): il riferimento `this`, copia e confronto di oggetti, membri `static`, variabili di istanza `final`, classi immutabili.

Homework 5 (da svolgere entro la prossima lezione)

Tre classi, ciascuna con il suo programma di prova (`ProvaRettangolo`, `ProvaOrario`, `ProvaLibretto`).

1. `Rettangolo.java`: base e altezza (`double`), sempre positive. Tre costruttori: senza argomenti (il rettangolo 1×1), con base e altezza (un valore non positivo diventa 1), di copia. Metodi `get`; metodi `set` che restituiscono `false`, senza modificare nulla, se il valore non è positivo; `area()`, `perimetro()` e il predicato `isQuadrato()`.
2. `Orario.java`: ore (0–23) e minuti (0–59). Costruttore `Orario(int o, int m)` (valori fuori intervallo: 00:00) e costruttore di copia; metodi `get`, nessun `set`. `avanza(int m)` sposta l'orario in avanti di `m` minuti, anche oltre la mezzanotte (23:50 più 20 minuti dà 00:10), e restituisce `false` se `m` è negativo; il predicato `precede(Orario altro)`; `descrizione()` restituisce una stringa come `"09:05"`. Il programma di prova legge ora di inizio e durata in minuti di un evento e ne stampa inizio e fine.
3. `Libretto.java`: i voti stanno in un `int[] private`, con una variabile che conta gli elementi in uso. Costruttore `Libretto(int capienza)`; `aggiungi(int voto)` restituisce `false` se il voto non è fra 18 e 30 o il libretto è pieno; `getQuanti()`, il predicato `isPieno()`, `media()` e `massimo()` (0 se vuoto). Nessun metodo restituisce l'array. Il programma di prova riceve i voti dalla riga di comando (`java ProvaLibretto 28 30 17 24`) e stampa quanti sono registrati, quanti rifiutati, la media con 2 decimali e il massimo.

Tutte le variabili di istanza sono `private`. Usate solo ciò che abbiamo visto finora.

Domande?

mattia.fonisto@unina.it